

Security & Sharing

How Salesforce controls who can see and edit which records and fields.

• Access Layers

Access is evaluated broad → narrow:

- 1. **Org** – login hours, IP ranges, MFA.
- 2. **Object** – CRUD via profiles / perm sets.
- 3. **Field** – FLS read / edit.
- 4. **Record** – OWD + sharing.

• Object Permissions

- Granted by **Profiles** and **Permission Sets**.
- CRUD + **View All** / **Modify All** for admin-style access.
- Object **View All** / **Modify All** ≠ org-wide **View All Data** / **Modify All Data**.
- Prefer **permission sets** over editing profiles.

• Field-Level Security

- Controls read / edit per field, per profile or perm set.
- Enforced in UI & API by default; in Apex only when you use `USER_MODE` or `stripInaccessible`.
- Inaccessible fields are then never returned to the client.

• Org-Wide Defaults

OWD	MEANING
Private	owner + role above
Public Read Only	all read, owner edits
Public Read/Write	all read + edit
Public R/W/Transfer	Leads & Cases only
Public Full Access	Campaigns only
Controlled by Parent	inherits master record

- Set the **most restrictive** baseline, then open up.

• Role Hierarchy

- Grants managers access to subordinates’ records.
- “Grant Access Using Hierarchies” opens vertical access.
- About record roll-up – not just an org chart.

• Sharing Rules

- Open access **laterally**, beyond the OWD baseline.
- **Owner-based** – by record owner’s role or group.
- **Criteria-based** – by field values on the record.
- Target roles, roles + subordinates, or public groups.

• Manual & Apex Sharing

- **Manual**: an owner shares one record (Sharing button).
- **Apex managed sharing**: create `AccountShare` rows with a reason.
- Programmatic and survives owner changes.

• Sharing in Apex

```
public with sharing class AcctSvc { }
public without sharing class Admin { }
public inherited sharing class Util { }
```

- `with sharing` enforces the running user’s **record** access.

• Enforcing in Apex

```
[SELECT Name FROM Account WITH USER_MODE];

Security.stripInaccessible(
    AccessType.READABLE, records);
```

- `WITH USER_MODE` enforces **CRUD + FLS + sharing** (all three layers).
- `WITH SECURITY_ENFORCED` enforced CRUD + FLS – removed in API v67+; use `USER_MODE`.

• Groups & Queues

- **Public Group** – reusable set of users, roles, groups.
- **Queue** – shared ownership + work routing.
- Use groups as sharing-rule targets for maintainability.

• Restriction & Scoping

- **Restriction rules** – hide shared records a user shouldn’t see.
- **Scoping rules** – set a user’s default record scope per object.

• Perm Set Groups

- Bundle permission sets for a **persona**.
- A **muting** permission set removes specific perms from the group.